

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]

Lecture #20: Logging Schemes

15-445/645 Database Systems (Fall 2019)

<https://15445.courses.cs.cmu.edu/fall2019/>

Carnegie Mellon University

Prof. Andy Pavlo

1 Crash Recovery

Recovery algorithms are techniques to ensure database consistency, transaction atomicity, and durability despite failures. DBMS is divided into different components based on the underlying storage device. We must also classify the different types of failures that the DBMS needs to handle.

Every recovery algorithm has two parts:

- Actions during normal transaction processing to ensure that the DBMS can recover from a failure.
- Actions after a failure to recover the database to a state that ensures atomicity, consistency, and durability.

The key primitives that we are going to use in a recovery algorithm are UNDO and REDO. Not all algorithms use both of these:

- **UNDO:** The process of removing the effects of an incomplete or aborted transaction.
- **REDO:** The process of re-instating the effects of a committed transaction for durability.

2 Failure Classification

Type #1: Transaction Failures

- **Logical Errors:** A transaction cannot complete due to some internal error condition (e.g., integrity, constraint violation).
- **Internal State Errors:** The DBMS must terminate an active transaction due to an error condition (e.g., deadlock)

Type #2: System Failures

- **Software Failure:** There is a problem with the DBMS implementation (e.g., uncaught divide-by-zero exception) and the system has to halt.
- **Hardware Failure:** The computer hosting the DBMS crashes. We assume that non-volatile storage contents are not corrupted by system crash.

Type #3: Storage Media Failure

- **Non-Repairable Hardware Failure:** A head crash or similar disk failure destroys all or parts of non-volatile storage. Destruction is assumed to be detectable. No DBMS can recover from this. Database must be restored from archived version

3 Buffer Pool Management Policies

Steal Policy: Whether the DBMS allows an uncommitted transaction to overwrite the most recent committed value of an object in non-volatile storage (can a transaction write uncommitted changes to disk).

- **STEAL**: is allowed
- **NO-STEAL**: is not allowed.

Force Policy: Whether the DBMS ensures that all updates made by a transaction are reflected on non-volatile storage before the transaction is allowed to commit

- **FORCE**: Is enforced
- **NO-FORCE**: Is not enforced

Force writes makes it easier to recover but results in poor runtime performance.

Easiest System to implement: NO-STEAL + FORCE

- The DBMS never has to undo changes of an aborted transaction because the changes were not written to disk.
- It also never has to redo changes of a committed transaction because all the changes are guaranteed to be written to disk at committed.
- **Limitation:** If all of the data that a transaction needs to modify does not fit on memory, then that transaction cannot execute because the DBMS is not allowed to write out dirty pages to disk before the transaction commits.

4 Shadow Paging

The DBMS maintains two separate copies of the database (**master, shadow**). Updates are only made in the shadow copy. When a transaction commits, atomically switch the shadow to become the new master. This is an example of a NO-STEAL + FORCE system.

Implementation:

- Organize the database pages in a tree structure where the root is a single disk page.
- There are two copies of the tree, the master and the shadow:
 - The root points to the master copy
 - Updates are applied to the shadow copy
- To install updates, overwrite the root so it points to the shadow, thereby swapping the master and shadow.
 - Before overwriting the root, none of the transactions updates are part of the disk-resident database.
 - After overwriting the root, all of the transactions updates are part of the disk resident database.
- **UNDO:** Remove the shadow pages. Leave master and the DB root pointer alone
- **REDO:** Not needed at all

5 Write-Ahead Logging

The DBMS records all the changes made to the database in a log file (on stable storage) before the change is made to a disk page. The log contains sufficient information to perform the necessary undo and redo actions to restore the database after a crash. This is an example of a STEAL + NO-FORCE system.

Almost every DBMS uses write-ahead logging (WAL) because it has the fastest runtime performance. But the DBMS's recovery time with WAL is slower than shadow paging because it has to replay the log.

Implementation:

- All log records pertaining to an updated page are written to non-volatile storage before the page itself is allowed to be overwritten in non-volatile storage.
- A transaction is not considered committed until all its log records have been written to stable storage.

- When the transaction starts, write a <BEGIN> record to the log for each transaction to mark its starting point.
- When a transaction finishes, write a <COMMIT> record to the log and make sure all log records are flushed before it returns an acknowledgment to the application.
- Each log entry contains information about the change to a single object:
 - Transaction ID.
 - Object ID.
 - Before Value (used for UNDO).
 - After Value (used for REDO).
- Log entries to disk should be done when transaction commits. You can use group commit to batch multiple log flushes together to amortize overhead.

6 Checkpoints

The main problem with write-ahead logging is that the log file will grow forever. After a crash, the DBMS has to replay the entire log, which can take a long time if the log file is large. Thus, the DBMS can periodically take a *checkpoint* where it flushes all buffers out to disk.

How often the DBMS should take a checkpoint depends on the application's performance and downtime requirements. Taking a checkpoint too often causes the DBMS's runtime performance to degrade. But waiting a long time between checkpoints can potentially be just as bad, as the system's recovery time after a restart increases.

Blocking Checkpoint Implementation:

- The DBMS stops accepting new transactions and waits for all active transactions to complete.
- Flush all log records and dirty blocks currently residing in main memory to stable storage.
- Write a <CHECKPOINT> entry to the log and flush to stable storage.

7 Logging Schemes

Physical Logging:

- Record the changes made to a specific location in the database
- Example: Position of a record in a page

Logical Logging:

- Record the high level operations executed by transactions. Not necessarily restricted to single page. Requires less data written in each log record than physical logging. Difficult to implement recovery with logical logging if you have concurrent transactions in a non-deterministic concurrency control scheme.
- Example: The UPDATE, DELETE, and INSERT queries invoked by a transaction.

Physiological Logging:

- Hybrid approach where log records target a single page but do not specify data organization of the page.
- Most commonly used approach.

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]